

AgriSense: Smart Irrigation & Machine Learning System

College Project Examination & Viva Preparation Guide

AGRISENSE VIVA STUDY GUIDE

1. System Architecture & Core Logic

AgriSense is a smart, IoT-enabled, closed-loop irrigation system. It consists of an ESP32 edge microcontroller connected to various physical sensors (soil moisture, temperature, flame detection, and GPS) that feeds data to a Flask Web Server hosted in the cloud. The web server runs a Machine Learning model (Random Forest Classifier) to make watering recommendations, checks external weather APIs to conserve water if rain is forecast, and stores data securely on a cloud-based serverless Postgres database (Neon) for real-time dashboard display.

2. Technology Stack Justification

- Python & Flask:

Flask is a lightweight micro-framework, making it extremely fast to set up and ideal for IoT APIs. We chose Python because the system runs a Machine Learning model (Random Forest). Writing the server in Python allows us to run the API and execute ML predictions in the same codebase natively.

- Neon PostgreSQL:

SQLite databases are file-based and get erased daily on Render's temporary containers. Neon Postgres provides secure, free, and permanent relational storage in the cloud. SQL is ideal because sensor telemetry is highly structured time-series data.

- HTML, CSS & JavaScript:

Standard web technologies keep the dashboard lightweight, fast to load, and easy to render dynamically using AJAX polling without client-side compilers like React.

- Render Hosting:

Render provides free cloud hosting connected directly to GitHub, automating deployment via Git commits and making it 24/7 accessible from anywhere.

3. Top 15 Predicted Viva Questions & Answers

Q1: What is the role of Machine Learning in this project?

Ans: Instead of using hardcoded rules (like 'turn pump on if moisture < 30%'), we use a Machine Learning Classifier. It analyzes moisture and temperature history to classify if irrigation is needed (1) or not (0), making the system smarter and adaptive to climate trends over time.

Q2: Why did you choose the Random Forest Classifier?

Ans: Random Forest is an ensemble learning method using multiple decision trees. It is highly robust, prevents overfitting, handles non-linear relationships between soil moisture and ambient temperature well, and has high classification accuracy with low compute requirements.

Q3: How does the 'Fallback Mode' work if a sensor fails?

Ans: If the soil moisture sensor returns out-of-bounds values (e.g. less than 0% or greater than 100%), the backend enters 'Fallback Mode'. It triggers an alert and uses historical telemetry and temperature degradation patterns to estimate the soil dryness, continuing irrigation safely.

AgriSense: Smart Irrigation & Machine Learning System

College Project Examination & Viva Preparation Guide

Q4: What is the purpose of the Open-Meteo Weather API integration?

Ans: To conserve water. If the ML model recommends turning the pump ON, the server queries the Open-Meteo forecast API using the station's GPS coordinates. If there is a >60% probability of rain in the next 24 hours, the server overrides the prediction and cancels the irrigation cycle.

Q5: Why did you use Neon Postgres instead of local SQLite?

Ans: Render's free tier uses temporary containers; any local SQLite file would be deleted when the container restarts daily. Neon Postgres runs in the cloud, keeping our user credentials, alert logs, and sensor history permanently saved.

Q6: How does the ESP32 connect and send data to the server?

Ans: The ESP32 connects to a Wi-Fi network using the WiFi.h library. It gathers sensor readings, formats them as a JSON payload, and executes an HTTP POST request to the server's public endpoint (/api/sensor) every 10 seconds.

Q7: How is user authentication secured on the website?

Ans: User credentials are secured using cryptographic password hashing (via Werkzeug's security helpers). The database never stores plain-text passwords, only hashes, protecting user accounts from data leaks.

Q8: What happens if the ESP32 goes offline?

Ans: The Flask server runs a background watchdog thread. If no telemetry is received from the ESP32 for more than 5 minutes, it flags the device status as 'Offline / Failing', writes an alert to the database, and sends an SMTP email notification to the operator.

Q9: How does the GPS module work and what is the fallback if it loses lock?

Ans: The Neo-6M GPS module feeds coordinates to the TinyGPS++ library on the ESP32. If there is no satellite lock (e.g. indoors), the ESP32 fails to send coordinates, and the server automatically falls back to geolocating the ESP32's connection IP via the IP-API service.

Q10: Why does the backend run background threads?

Ans: We use Python's threading library to prevent blocking. Tasks like reverse geocoding, sending email alerts, and retraining the ML model take time and would slow down the API response if run synchronously. Threading keeps the system highly responsive.

Q11: What baud rates are used in the ESP32 hardware configuration?

Ans: We use 115200 bps for the primary Hardware Serial 0 interface (used for PC debugging) and 9600 bps for the Hardware Serial 2 interface (used to communicate with the Neo-6M GPS module).

Q12: How does the soil moisture sensor map analog values to percentage?

Ans: The soil moisture sensor is an analog probe. We calibrated it by reading raw values: 4095 in dry air (0%) and 1500 fully wet in water (100%). The ESP32 code maps these raw readings into a standard 0 to 100% scale using the map() function.

Q13: What happens when the dashboard receives a fire warning?

Ans: If the flame sensor registers heat, the server overrides all logic, turns the pump ON immediately to extinguish the fire, flags a Critical Fire warning on the UI, and dispatches an urgent email alert via SMTP.

AgriSense: Smart Irrigation & Machine Learning System

College Project Examination & Viva Preparation Guide

Q14: What is the role of gunicorn in the Render deployment?

Ans: Flask's built-in server is only for local development and is single-threaded. Gunicorn (Green Unicorn) is a production-grade Python WSGI HTTP server that manages multiple worker processes to handle web traffic efficiently in the cloud.

Q15: How does the frontend dashboard get live updates?

Ans: A JavaScript polling interval (in main.js) executes an AJAX GET request to the backend's /api/live_data endpoint every few seconds. It parses the JSON response and updates the UI DOM and charts dynamically without reloading.